collector (GC) runs are moved to the old generation space. Providing a larger heap for Elasticsearch helps avoid filling up the young generation space quickly, subsequently avoiding garbage collection runs.

A general rule of thumb is to provide at least half of the memory as heap memory to the application. For example, if your machine is configured with 16 GB RAM, be sure the heap is set to at least 8 GB. We can set this by configuring the .options file with the `-Xmx` setting (see section 14.5.3).

Also, attaching a local storage disk enables better performance when writing to disk rather than using a network-based filesystem and disk. Ultimately, provisioning individual nodes with local volumes is a better strategy.

Remember that we must understand the indexing and shard requirements to allocate initial memory. For example, based on how many shards and replicas we intend to hold on the node, we may need to calculate and provide physical disk memory.

Optimal shard numbers help avoid building a cluster with too many small shards. As the number of shards increases, so does communication between the nodes. Thus, performance also relies on network capacity and bandwidth. Adding replicas increases search query read throughput, but having too many replicas quickly consumes memory, and a larger number of replicas is difficult to manage.

15.1.2 Document modeling

Elasticsearch is a NoSQL database; data should be denormalized, unlike in relational databases, where data is normalized. For example, if an employee is created, the record consists of the full information about the employee.

Every document in Elasticsearch is self-contained, so no joins are expected on the data. If your data is predominantly parent-child, you may want to rethink using Elasticsearch. Nested and parent-child operations are slow and degrade performance from the outset. If you come from a relational database world, be sure you understand the NoSQL data modeling principles.

In addition, we should aim to limit searching multiple fields because searching queries across multiple fields slows the query response time. Instead, combine multiple fields into a single field and search against this single field. Fortunately, Elasticsearch provides a `copy_to` attribute on fields, which helps in this scenario.

Let's look at an example. Suppose we have a couple of fields (`title` and `synopsis`) in our `book` document, and they are indexed in our `programming_books1` index. When we write queries to search in `title` and `synopsis`, we usually search against these two individual fields.

Listing 15.1 A non-performant search across multiple fields

```
PUT programming_books1/_doc/1
{
  "title":"Elasticsearch in Action",
  "synopsis":"A straightforward, hands-on, example driven, action book"
}
```

Indexes a sample document before issuing a search